

CprE 381: Computer Organization and Assembly-Level Programming

Project Part 3 Report

Team Members: **Caden Otis and Christopher Hausner**

Project Teams Group #: **B_05**

1. Introduction:

This report will cover the last project of CPRE 3810, which will focus on comparing the performance of the three MIPS-based processors we've developed in the class. The first processor is the single-cycle processor, which executes one MIPS instruction every cycle and has a CPI of 1.0. The next processor is the software-scheduled processor, which is a pipelined processor that avoids data and control hazards by inserting NOPs in assembly test files but raises the average CPI. The third processor is the hardware-scheduled processor, which is another pipelined processor that uses stalling and forwarding to avoid data and control hazards while also introducing a low ideal CPI of 1. In this project, we will analyze the performance of each processor, consider SW and HW optimizations that we could've made to these processors, and write programs that take in these optimizations. We will then reflect on the challenges that we faced while making these three processors and how we got around them.

2. Benchmarking

Single Cycle Processor

Program Run	# Instructions	Total Cycles	CPI	Max Cycle Time	Execution Time
Proj1_base_tests	27	27	1	48.12 ns	1.29 μ s
Grendel	2116	2116	1	48.12 ns	101.82 μ s
Proj1_bubbleSort	1144	1144	1	48.12 ns	55.04 μ s

Software Pipelined Cycle Processor

Program Run	# Instructions	Total Cycles	CPI	Max Cycle Time	Execution Time
Proj2_base_tests	99	115	1.16	18.51 ns	2.12 μ s
Grendel2	5472	5990	1.09	18.51 ns	110.40 μ s
Proj2_bubbleSort	3134	3462	1.1	18.51 ns	63.81 μ s

Hardware Pipelined Cycle Processor

Program Run	# Instructions	Total Cycles	CPI	Max Cycle Time	Execution Time
Proj1_base_tests	27	31	1.15	24.36 ns	0.75 μ s
Grendel	2116	3024	1.43	24.36 ns	73.71 μ s
Proj1_bubbleSort	1144	1577	1.38	24.36 ns	38.45 μ s

CPI was calculated as $\text{TotalCycles}/\text{\#Instructions}$. Then Max Cycle time was $1/\text{MaxFrequency}$. Execution time was calculated by doing $\text{\#Inst} * \text{CPI} * \text{MaxCycleTime}$. #Instructions came from Mars, Total cycles came from the testbench. With this knowledge it is important to point out some certain details. Our Hardware Pipeline will stall for only 1 data hazard, when there is a load and then it is accessed and it will only stall 1 cycle. Then for control hazards, if we are jumping or we are branching it will flush 2 cycles. Our software pipeline has no bypass in the register file, thus for data hazards it needs 3 NOPS. Finally, our single cycle design has no hazards as one would expect for single cycle nature.

It can be seen that Grendel has almost double the # instructions of bubble sort, but grendel takes less time than double bubble sort. This is due to the instruction types that are performed. Any branch or jump instruction taken is going to increase our Cycles by 3, due to flushing. As a result I would estimate that the percentage of jumps/branches is higher in bubble sort than it is in grendel, in relation to jumps/branches percent out of # instructions. Sure enough, testing in MARS shows that 21% of my instructions are jumps/branches in bubble sort, and yet in grendel only 15% are branches and jumps. This explains why nearly double the instructions of bubble sort in grendel's instruction count, is less than double the time for 2 bubble sorts.

In relation to the single cycle, this is when the single cycle has a chance to compete, when there are lots of branches and jumps. This allows the single cycle to do these in 1 cycle while the hardware and software pipelines need 2 and 3 respectively. In an application with a lot of jumps you could argue the single cycle would be better but unless it is majority jumps, the hardware pipeline will still outperform the single cycle.

Our performance was best on our Hardware Pipelined processor, and worse on our software pipelined processor in terms of execution time. Our cycle time is best on our software Pipeline but given all of the NOPS that must be added in to avoid hazards, our instruction could more than doubles for every program causing the execution time to be more.

To start with Base Tests, our Hardware pipeline performed best. It had a CPI of 1.15 which is more than our single cycle design, yet given the reduced cycle time and pipeline nature, we have over a 25% increase in execution time. We only have 4 more cycles which is not a lot given our increased cycle time. Our software pipeline however performed worse given we must add NOPS which increases our instruction count significantly to 99. This means the hardware pipeline is over 2x faster than the software pipeline.

Moving on to grendel we see similar results. The software pipeline has 5472 instructions due to NOPS inserted which would be 2116 without NOPS. Our CPI is higher on our hardware pipeline due to the branch and jumps in the program, increasing it to 1.43 which is much higher than our software at 1.09 and our single cycle at 1.0 as always. Our total cycles on our single cycle is 2116 as expected, while our

hardware pipeline has a whopping 3024 cycles. However, still our hardware pipeline performance is better due to the lower cycle time.

Finally, onto bubble sort our hardware pipeline yet again performs best. The software pipeline has over double the instructions of the single cycle and hardware, due to 3 NOPS needed at data hazards. Our hardware pipeline has over 400 more cycles than our single cycle design, but once again due to our low cycle time will perform better. These increased cycles are due to stalling for control hazards. Without data hazard forwarding our hardware pipeline would not compete with the single cycle design, but with forwarding for data hazards we can see that our hardware pipelined design overtakes the single cycle design in performance. Our software pipeline is not efficient enough to be better than our single cycle due to the tremendous amount of NOPS required to work, it essentially makes the idea of pipeline worse than keeping a single cycle processor. If we were to avoid data hazards our software pipeline would perform better than the single cycle, however this is not always practical.

As for our hardware pipeline, the only case it would not be better than our single cycle is if we are doing a bunch of jumps and branches more than we are data hazards. When we jump or branch we have to flush, essentially wasting 2 cycles. This would mean the one instruction (+ flush cycles) would take over 70 ns. So for example if we had a program with 100 branches/jumps and 10 instructions that are not branches/jumps, our single cycle processor would outperform our hardware pipelined processor due to not needing to stall when branching or jumping. In our code examples though, we do not jump and branch very often (relatively) which allows our hardware pipeline to perform best.

4. Software Optimization:

One software optimization that we could make to the software-scheduled processor is to be a lot more conservative on how we used NOPs with our assembly files. Whenever a control hazard occurred, we would insert 4 NOP instructions right after a branch or jump instruction to ensure that no incorrect instructions were fetched and messing up the control flow execution of instructions. By using 4 NOPs for every branch or jump instruction, we are stalling more than necessary, which leads to more cycles being executed for these instructions and increasing the average CPI to result in poor execution time/performance. By using less NOP instructions, there would be less total stalls occurring during the execution of our assembly files, which would decrease the average CPI and improve overall performance.

To be more conservative with our NOP usage, we could've used three NOPs after every branch or jump instruction to avoid control hazards. This is because we really only needed to stall for the IF, ID, and EX stages that handle updating the PC with the jump or branch target address. By having less NOPs being executed per control hazard, the total instruction count will be much smaller, reducing the execution time and improving performance.

If we look at the execution time of `grendel.s` executing on the software-scheduled processor, it originally has an execution time of 110.40 μ s. There are about 18 jump instructions and 10 branch instructions that use 4 NOP instructions. If we instead change them to use two NOP instructions, then there will be $5472 - (18 + 10) = 5444$ total instructions executed instead. This would result in an execution time of:

$$5444 * 1.09 * 18.51 = 109.84 \mu\text{s}$$

$$\text{Speedup} = 110.40 / 109.84 = 1.01$$

This optimization results in a speedup of 1.01. In other words, this optimization improves the execution time of `grendel.s` by 1%.

5. Hardware Optimization

Single-cycle optimization:

To improve our single-cycle processor design, one optimization that we could make is change the shifter module that exists in the ALU. The shifter is on the critical path for this processor and adds more delay to the ALU. To improve the shifter, we could split the shifter functionality into three separate components instead of just having one large shifter component that includes all of the shifting functionality. By splitting the shifting component up into several components, the critical path won't have to go through as many MUXes every single time a shift operation needs to be performed within the circuit. By changing and optimizing our shifter, we can compensate for the amount of propagation delay the instruction and data memory components have, which will result in a better slack time and better frequency performance.

To implement this change, we would create three separate shifter modules for each of the different shifting instructions: sll, srl, sra. These shifter modules would be similar to our current shifter module, but it wouldn't need an additional 2-bit input to indicate the shifting type. This change would also allow for the new shifter module to have 5 less MUXes implemented into the design. This is because it doesn't have to worry about which direction to shift the value (left or right and zero- or sign-extend) because each shifter will be for a dedicated shifting type. We would also have these new shifter modules outside of the ALU and run in parallel with the ALU. By minimizing the shifting amount and taking the shifting functionality out of the ALU, this would greatly increase the speed of the ALU and shorten our critical path.

In the synthesis results for our single-cycle processor, 4.35 ns of the critical path is spent within the shifter module of the ALU. If we use our new implementation previously discussed, then that time will be eliminated from the ALU since the shifting will be done outside of the ALU. This would result in a slightly faster cycle time and reduce the overall execution time by about 5% - 8%.

SW-scheduled optimization:

To improve our software-scheduled processor, we could implement dynamic branch prediction to store if branches are taken or not into a buffer and make a prediction on the result of the next branch. This would reduce the amount of stalling that we do currently for each branch instruction, where every branch instruction is stalled for four cycles. If we decided to change our branch logic to include a 2-bit predictor, then we could more accurately predict if a branch will be taken or not and only stall when necessary. If a branch prediction is wrong, then there would only be a 1-cycle penalty to backtrack and flush out the incorrectly fetched instruction. With this optimization, this would also eliminate jump stalls as well because the buffer would be able to accurately predict that jumps are always taken.

To implement this change, we would need to create a branch target buffer component that would store each branch outcome and help to predict the next branch behavior. This buffer would be placed in the IF stage and take the next instruction's memory address from the PC as its sole input. The buffer would then be able to calculate the predicted branch target address by using the stored target addresses and finding the correct branch target address from the PC address. If a branch is predicted to be taken, the branch target address can be fetched right away. Same logic applies for jump instructions.

If we look at the execution time of `grendel.s` executing on the software-scheduled processor, it originally has an execution time of 110.40 μ s. There are about 18 jump instructions and 10 branch instructions that use 4 NOP instructions. If we add branch prediction logic and assume that 7 out of 10 branch instructions are predictably correct then there will be $5472 - (4 * (18 + 7)) + 3$ 1-cycle penalties = 5375 total instructions executed instead. This would result in an execution time of:

$$5375 * 1.09 * 18.51 = 108.45 \mu\text{s}$$

$$\text{Speedup} = 110.40 / 108.45 = \mathbf{1.02}$$

This optimization results in a speedup of 1.02. In other words, this optimization improves the execution time of `grendel.s` by **2%**.

HW-scheduled optimization:

To improve our hardware-scheduled processor, we can improve how our branch logic works in the EXE stage of our pipeline. Our current branch logic can't execute until after the ALU finishes executing because it depends on the `s_ALUZero` output from the ALU. Because of this, the propagation delay of the EXE stage is longer than expected. Instead, we could move the branch logic, more specifically the Adder output that determines if we need to branch (`s_Branching`), to propagate to the MEM stage. By doing this, the EXE stage will take less time to execute since the branch logic will be ultimately done in the MEM stage. This will then allow for the branch logic to be executed in parallel with Data Memory in the MEM stage and not have to wait for the ALU, reducing the branch logic latency on the critical path.

To implement this change, all that would need to be done is have the ALU output `s_ALUZero` and Adder output `s_Branching` be inputs and outputs of the EX/MEM pipeline register. Then the AND gate that sets the `s_NotJump` select input would be moved to the MEM stage, while still propagating its output to the IF stage. This implementation change should decrease the propagation delay of the EXE stage while not increasing the propagation delay of the MEM stage, therefore reducing our critical path.

In the synthesis results for our hardware-scheduled processor, about 2 ns of its critical path is spent within the AND gate that sets the `s_NotJump` select input to say whether a jump has or hasn't occurred. If we use our new implementation previously discussed, that time can be eliminated and will be moved to the MEM where it can be done right away once in the MEM stage. This would result in a slightly faster cycle time and reduce the overall execution time by about 2% - 5%, very similar to the single-cycle optimization performance improvement.

6. It Depends

As we touched on earlier, our single cycle processor will only outperform the hardware pipeline processor when there are lots of jumps or branches taken. Our hardware pipeline flushes 2 cycles when we jump or take a branch, and stalls 1 cycle when we are accessing a load right after. To show this, I wrote a simple small code. Here it is below:

```
addi $s2, $0, 20
j begining
jump2:
addi $s1, $s1, -1
j begining
jump1:
beq $s2, $s1, exit
j jump2
begining:
addi $s1, $s1, 2
j jump1
exit:
Halt
```

This program consists mainly of Jumps. As a result this is what i found

Single Cycle:

CPI = 1

Inst = 114

Time per Cycle = 48.12 ns

Execution time = 5.48 μ s

Hardware Pipeline:

CPI = 2.04

Inst = 114

Time per Cycle = 24.36 ns

Execution time = 5.66 μ s

As you can see, the single cycle processor performs better. The flushing overweighs the CPI and makes not as fast. This will depend on the software however, depending on the percent of jumps/branches taken. If we added one more instruction on one of the loops of the program that was not a jump or a branch, then the multicycle processor would be better, ex: add a `addiu $s3, $s3, 1` after `jump2`:. This would result in our processor ET being 6.35 μ s for the single cycle and 6.07 μ s. The case on the single cycle out performing our multicycle is very rare, and honestly not practical at all.

As for the hardware pipeline outperforming our software pipeline, this is always going to be the case unless the compiler does some funky stuff I will touch on later. Our software pipeline does not have register bypassing - data hazard need 3 nops to go to WB and then be usable. We must stall on the software pipeline at all data hazards, all jumps, all branches regardless of taken or not (cause we don't know if its taken or not). Our hardware pipeline will only stall when accessing a register that was just loaded into and only needs 1 stall, aka load use hazard. So for all common use cases, and practically most programs our hardware pipeline will outperform our software pipeline by quite a substantial amount. This can be seen in the examples in the table on the first page.

The only case where the software pipeline will outperform, is if the compiler will reshuffle instructions for the software pipeline - but not for the hardware pipeline. If this was the case, the software pipeline can execute instructions in replacement of NOPS after branching, while the hardware pipeline will have to do NOPS due to the hardware implementation. This is not practical however, and would likely not be the case due to a very complex compiler needed. So we can safely say that the hardware pipeline will outperform the software pipeline. If there are no data dependencies then they could be equal with each other but data dependencies are common, and not practical to assume there are none. The best the software pipeline can do is match the hardware. Thus, our software pipeline will (almost assuming the compiler doesn't reshuffle) never outperform the hardware pipeline.

7. Challenges

One of the challenges that we faced during this term project was when we finished implementing our software-scheduled pipeline design and assembly test files and were ready to test our design. When testing our design, we were having tests fail and we weren't sure if it was because of the test itself or our processor design, which made it confusing to know what the issue was and how to move forward. We eventually asked a TA to look at our assembly test files since that would be the easiest thing to fix and try to narrow down the potential causes for our test failures, and the TA found some minor mistakes in our test files. Once fixing those, all tests that we created passed with our software-scheduled processor design. We could avoid this challenge in the future by being more thorough in our test benches and creating a test bench for each module and functionality of our processor design. That way, we can be sure our processor works perfectly when we test it with assembly test files, and we'd know the test is at fault if a test file ended up failing.

Another challenge that we faced during this term project was implementing flushing as a part of avoiding control hazards. The way that we implemented flushing was by using it as a part of the reset signal that gets inputted into the different pipeline registers. However, since these reset signals are asynchronous, this would cause flushing to occur unexpectedly and give us incorrect results. We were able to fix this issue by changing the reset signal to be synchronous in the pipeline registers, where only flushing would occur at the next positive edge of the clock. We could avoid this challenge in the future by taking more time to design a solution that we fully understand. We somewhat quickly came up with our design to flush instructions out of certain stages by following the reset signal, but we didn't pay attention to the fact that the reset signal is asynchronous. If we slowed down and paid attention to this fact, we could've avoided this entire issue.

Another challenge that we faced during this term project was how we went about our version control and sharing code implementations with everyone in the group. Our version control for parts 1 and 2 of the term project was to just upload files (VHDL, MIPS, etc,) to a shared folder in Cybox. We decided to go with Cybox because we weren't show how to incorporate Git into this project. We thought it would take too long to set up Git and would just be confusing, so we used Cybox instead for sharing files. However, using Cybox made it tricky for us to understand what changes were made to files when they were uploaded. This also made it easy for files to accidentally get deleted and then not be able to be recovered. However, after some time, we learned how to use Cybox effectively and share code changes with one another in the group. Looking back, we could've used GitHub instead to be able to easily see who did what and have a committed history of all code changes. This would've been extremely useful when cases like if a file accidentally got overridden, which did happen to us.